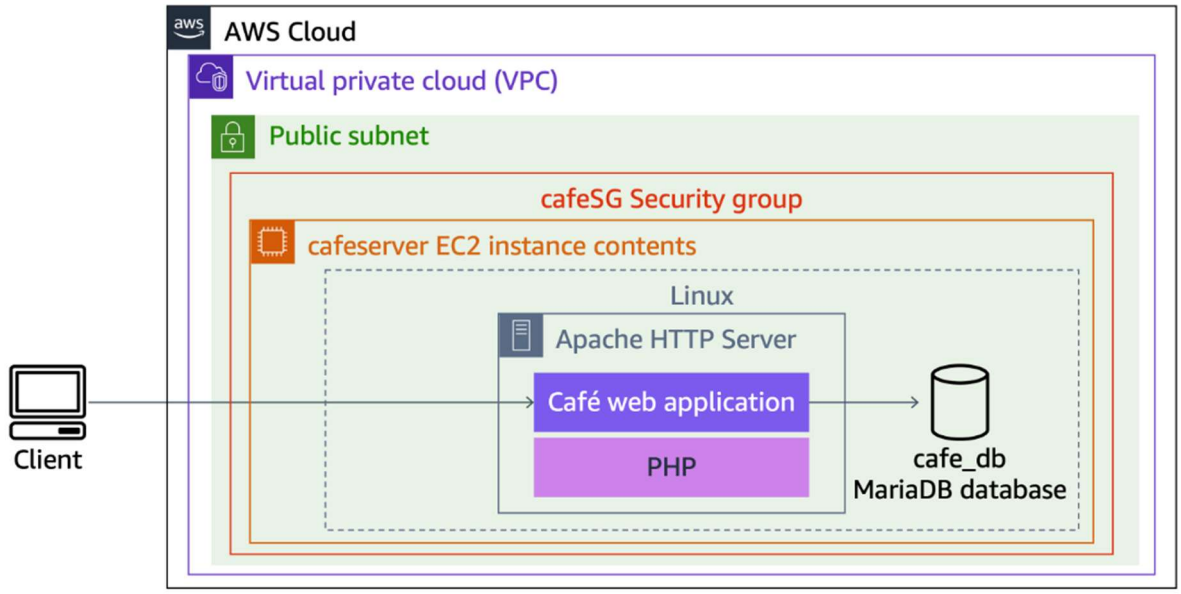
173-[JAWS]-Activity: Troubleshoot Create Instance

En esta práctica se muestra como lanzar una instancia EC2 desde la línea de comandos de AWS y como solucionar problemas de despliegue del sitio web.

Escenario

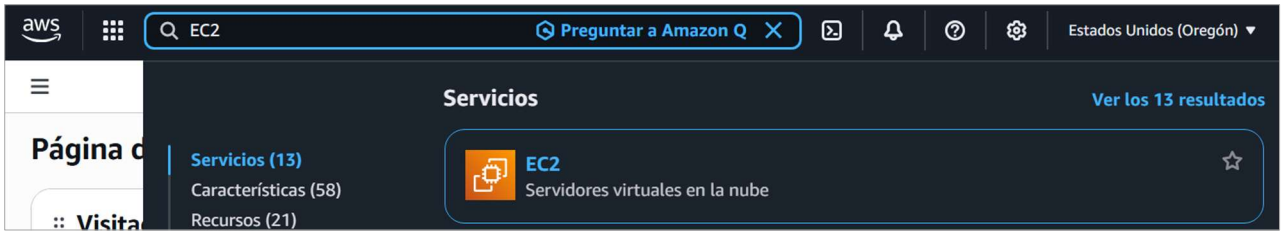
La arquitectura final es la siguiente:



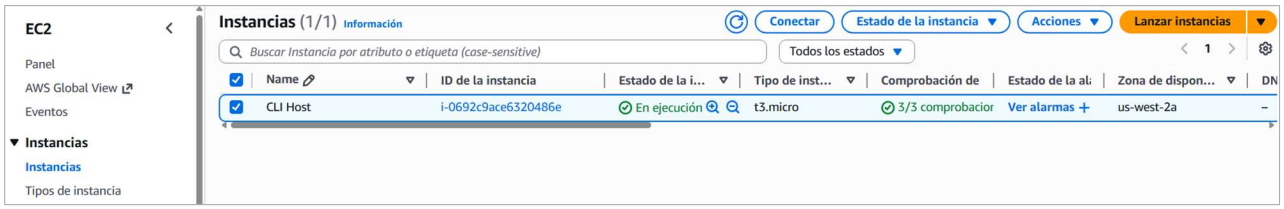
Este es un ejemplo en el que la base de datos está en la misma instancia, no en una RDS

Conectar a la instancia *CLI Host*

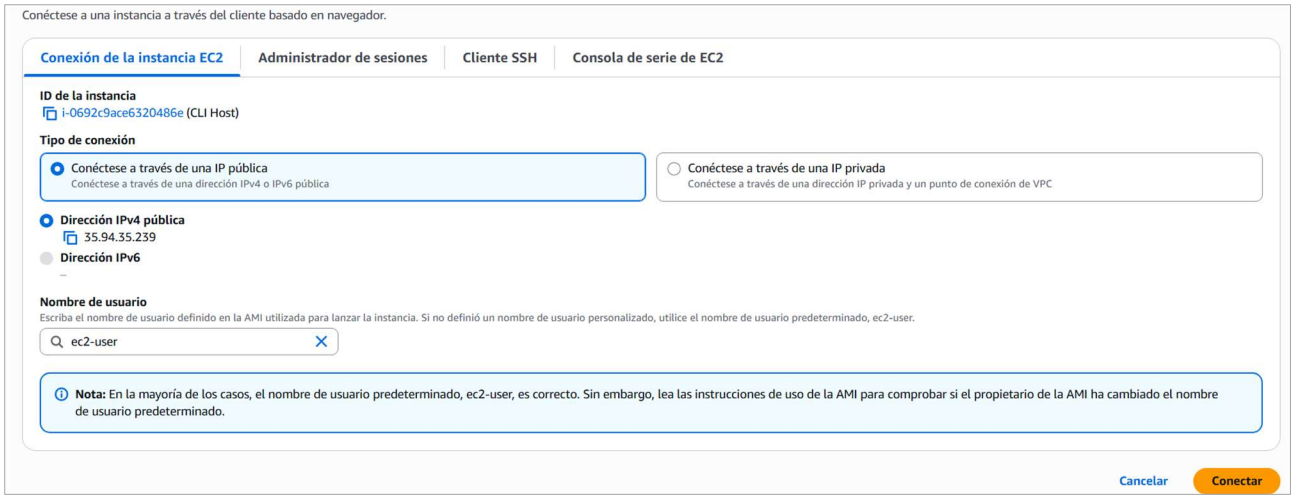
Abrimos el panel de *EC2*



Seleccionamos la instancia *CLI Host* y hacemos clic en *Conectar*



Nos conectamos por *Instance Connect*



Configurar AWS CLI

Las instancias con Linux ya tienen el AWS CLI preinstalado.

Ingresamos el AWS Access Key ID, el AWS Secret Access Key y la región

```
[ec2-user@cli-host ~]$ aws configure
AWS Access Key ID [None]: AKIA2BBHGPTS2ROH32XC
AWS Secret Access Key [None]: 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
Default region name [None]: us-west-2
Default output format [None]: json
```

Crear una instancia EC2 mediante AWS CLI

El archivo *create-lamp-instance-v2.sh* contiene el script para lanzar la instancia.

Creamos un respaldo de este archivo

```
[ec2-user@cli-host ~]$ cd ~/sysops-activity-files/starters
[ec2-user@cli-host starters]$ cp create-lamp-instance-v2.sh create-lamp-instance.backup
```

Checamos el contenido del archivo usando el editor Vi

```
[ec2-user@cli-host starters]$ view create-lamp-instance-v2.sh
```

El comando *view* es una variante del editor de texto Vim que abre un archivo en modo de solo lectura.

Podemos mostrar el número de líneas tecleando *:set number* + *Enter*

```
:set number
```

i-0692c9ace6320486e (CLI Host)

PublicIPs: 35.94.35.239 PrivateIPs: 10.200.0.245

En esta parte del script se indica el tipo de instancia como t3.small

```
1 #!/bin/bash
2 DATE=`date '+%Y-%m-%d %H:%M:%S'`
3 echo
4 echo "Running create-instance.sh on "$DATE
5 echo
6
7 # Hard coded values
8 instanceType="t3.small"
9 echo "Instance Type: "$instanceType
10 profile="default"
11 echo "Profile: "$profile
12
13 echo
14 echo "Looking up account values..."
15
```

Itera por cada una de las regiones de AWS hasta encontrar la VPC con el nombre "Café VPC" y obtiene el ID de la VPC

```
16 # get vpcId
17 vpc=""
18 while [[ "$vpc" == "" ]] ; do
19   for i in $(aws ec2 describe-regions | grep RegionName | cut -d '"' -f4) ; do
20     region=$i;
21     vpc=$(aws ec2 describe-vpcs --region $i --filters "Name=tag:Name,Values='Cafe VPC'" --profile $profile | grep VpcId | cut -d '"' -f4 | sed -n 1p );
22     if [[ "$vpc" != "" ]]; then
23         break;
24     fi
25   done
26 done
27 echo
28 echo "VPC: "$vpc
29 echo "Region: "$region
```

Posteriormente busca en la VPC la subred con el nombre "Cafe Public Subnet 1" y obtiene el ID de la subred.

Obtiene el ID de la AMI que se usará para lanzar la instancia, con la versión más reciente de Linux

```
31 vpc=$(aws ec2 describe-vpcs \
32 --filters "Name=tag:Name,Values='Cafe VPC'" \
33 --region $region \
34 --profile $profile | grep VpcId | cut -d '"' -f4 | sed -n 1p)
35 echo "VPC: "$vpc
36
37 # get subnetId
38 subnetId=$(aws ec2 describe-subnets \
39 --filters "Name=tag:Name,Values='Cafe Public Subnet 1'" \
40 --region $region \
41 --profile $profile \
42 --query "Subnets[*]" | grep SubnetId | cut -d '"' -f4 | sed -n 1p)
43 echo "Subnet Id: "$subnetId
44
45 # Get keypair name
46 key=$(aws ec2 describe-key-pairs \
47 --profile $profile --region $region | grep KeyName | cut -d '"' -f4 )
48 echo "Key: "$key
49
50 # Get AMI ID
51 imageId=$(aws ssm get-parameters \
52 --names '/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2' \
53 --profile $profile \
54 --region $region | grep ami- | cut -d '"' -f4 | sed -n 2p)
55 echo "AMI ID: "$imageId
```

Después verifica si la instancia "cafeserver" ya existe en la cuenta de AWS. De ser así pregunta al usuario si desea borrarla.

```
57 #check for existing cafe instance
58 existingEc2Instance=$(aws ec2 describe-instances \
59 --region $region \
60 --profile $profile \
61 --filters "Name=tag:Name,Values=cafeserver" "Name=instance-state-name,Values=running" \
62 | grep InstanceId | cut -d '"' -f4)
63 if [[ "$existingEc2Instance" != "" ]]; then
64   echo
65   echo "WARNING: Found existing running EC2 instance with instance ID "$existingEc2Instance"."
66   echo "This script will not succeed if it already exists. "
67   echo "Would you like to delete it? [Y/N]"
68   echo ">>"
69
70   validResp=0
71   while [ $validResp -eq 0 ];
72   do
73       read answer
74       if [[ "$answer" == "Y" || "$answer" == "y" ]]; then
75           echo
76           echo "Deleting the existing instance..."
77           aws ec2 terminate-instances --instance-ids $existingEc2Instance --region $region --profile $profile
78           #wait for confirmation it was terminated
79           aws ec2 wait instance-terminated --instance-ids $existingEc2Instance --region $region --profile $profile
80           validResp="1"
```

También busca si existe un grupo de seguridad que en el nombre contenga la cadena "cafeSG". De ser así pregunta al usuario si desea borrarlo.

```
81        elif [[ "$answer" == "N" || "$answer" == "n" ]]; then
82            echo "Ok, exiting."
83            exit 1
84        else
85            echo "Please reply with Y or N."
86        fi
87    done
88
89    sleep 10 #give it 10 seconds before trying to delete the SG this instance used.
90  fi
91
92  #check for existing cafeSG security Group
93  existingMpSg=$(aws ec2 describe-security-groups \
94  --region $region \
95  --query "SecurityGroups[?contains(GroupName, 'cafeSG')]" \
96  --profile $profile | grep GroupId | cut -d '"' -f4)
97
98  if [[ "$existingMpSg" != "" ]]; then
99    echo
100   echo "WARNING: Found existing security group with name "$existingMpSg"."
101   echo "This script will not succeed if it already exists. "
102   echo "Would you like to delete it? [Y/N]"
103   echo ">>"
104
105   validResp=0
106   while [ $validResp -eq 0 ];
```

```
107   do
108       read answer
109       if [[ "$answer" == "Y" || "$answer" == "y" ]]; then
110           echo
111           echo "Deleting the existing security group..."
112           aws ec2 delete-security-group --group-id $existingMpSg --region $region --profile $profile
113           validResp="1"
114       elif [[ "$answer" == "N" || "$answer" == "n" ]]; then
115           echo "Ok, exiting."
116           exit 1
117       else
118           echo "Please reply with Y or N."
119       fi
120   done
121   sleep 10 #give it 10 seconds before trying to recreate the SG
122 fi
```

Se crea el grupo de seguridad “cafeSG” permitiendo el acceso por el puerto 22 y el puerto 8080

```
124 # CREATE a security group and capture the name of it
125 echo
126 echo "Creating a new security group..."
127 securityGroup=$(aws ec2 create-security-group --group-name "cafeSG" \
128 --description "cafeSG" \
129 --region $region \
130 --group-name "cafeSG" \
131 --vpc-id $vpc --profile $profile | grep GroupId | cut -d '"' -f4 )
132 echo "Security Group: "$securityGroup
```

```
# Open ports in the security group
echo
echo "Opening port 22 in the new security group"
aws ec2 authorize-security-group-ingress \
--group-id $securityGroup \
--protocol tcp \
--port 22 \
--cidr 0.0.0.0/0 \
--region $region \
--profile $profile

echo "Opening port 80 in the new security group"
aws ec2 authorize-security-group-ingress \
--group-id $securityGroup \
--protocol tcp \
--port 8080 \
--cidr 0.0.0.0/0 \
--region $region \
--profile $profile
```

Se lanza la instancia con los parámetros creados anteriormente

Se indica que los datos de usuario están en el archivo create-lamp-instance-userdata-v2.txt

```
echo
echo "Creating an EC2 instance in "$region
instanceDetails=$(aws ec2 run-instances \
--image-id $imageId \
--count 1 \
--instance-type $instanceType \
--region us-east-1 \
--subnet-id $subnetId \
--security-group-ids $securityGroup \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=cafeserver}]' \
--associate-public-ip-address \
--iam-instance-profile Name=LabInstanceProfile \
--profile $profile \
--user-data file://create-lamp-instance-userdata-v2.txt \
--key-name $key )
```

Se muestra el detalle de la instancia usando json.tool de Python y se imprime la dirección IP pública

```
#if the create instance command failed, exit this script
if [[ "$?" -ne "0" ]]; then
  exit 1
fi

echo
echo "Instance Details...."
echo $instanceDetails | python -m json.tool

# Extract instanceId
instanceId=$(echo $instanceDetails | python -m json.tool | grep InstanceId | sed -n 1p | cut -d '"' -f4)
echo "instanceId="$instanceId
echo
echo "Waiting for a public IP for the new instance..."
pubIp=""
while [[ "$pubIp" == "" ]]; do
  sleep 10;
  pubIp=$(aws ec2 describe-instances --instance-id $instanceId --region $region --profile $profile | grep PublicIp | sed -n 1p | cut -d '"' -f4)
done
```

```
echo
echo "The public IP of your LAMP instance is: "$pubIp
echo
echo "Download the Key Pair from the Vocareum page."
echo
echo "Then connect using this command (with .pem or .ppk added to the end of the keypair name):"
echo "ssh -i path-to/"$key" ec2-user@"$pubIp
echo
echo "The website should also become available at"
echo "http://"$pubIp"/cafe/"

echo
DATE=`date '+%Y-%m-%d %H:%M:%S'`
echo
echo "Done running create-instance.sh at "$DATE
echo
```

Tecleamos *:q!* + *Enter* para salir

El script completo es el siguiente:

```
#!/bin/bash
DATE=`date '+%Y-%m-%d %H:%M:%S'`
echo
echo "Running create-instance.sh on "$DATE
echo

# Hard coded values
instanceType="t3.small"
echo "Instance Type: "$instanceType
profile="default"
echo "Profile: "$profile

echo
echo "Looking up account values..."

# get vpcId
vpc=""
while [[ "$vpc" == "" ]] ; do
  for i in $(aws ec2 describe-regions | grep RegionName | cut -d "" -f4) ; do
    region=$i;
    vpc=$(aws ec2 describe-vpcs --region $i --filters "Name=tag:Name,Values='Cafe
VPC'" --profile $profile | grep VpcId | cut -d "" -f4 | sed -n 1p );
    if [[ "$vpc" != "" ]]; then
        break;
    fi
  done
done
echo
echo "VPC: "$vpc
echo "Region: "$region
```

```
vpc=$(aws ec2 describe-vpcs \
--filters "Name=tag:Name,Values='Cafe VPC'" \
--region $region \
--profile $profile | grep VpcId | cut -d "" -f4 | sed -n 1p)
echo "VPC: "$vpc

# get subnetId
subnetId=$(aws ec2 describe-subnets \
--filters "Name=tag:Name,Values='Cafe Public Subnet 1'" \
--region $region \
--profile $profile \
--query "Subnets[*]" | grep SubnetId | cut -d "" -f4 | sed -n 1p)
echo "Subnet Id: "$subnetId

# Get keypair name
key=$(aws ec2 describe-key-pairs \
--profile $profile --region $region | grep KeyName | cut -d "" -f4 )
echo "Key: "$key

# Get AMI ID
imageId=$(aws ssm get-parameters \
--names '/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2' \
--profile $profile \
--region $region | grep ami- | cut -d "" -f4 | sed -n 2p)
echo "AMI ID: "$imageId

#check for existing cafe instance
existingEc2Instance=$(aws ec2 describe-instances \
--region $region \
--profile $profile \
--filters "Name=tag:Name,Values=cafeserver" "Name=instance-state-name,Values=running" \
| grep InstanceId | cut -d "" -f4)
if [[ "$existingEc2Instance" != "" ]]; then
  echo
  echo "WARNING: Found existing running EC2 instance with instance ID "$existingEc2Instance"."
  echo "This script will not succeed if it already exists. "
  echo "Would you like to delete it? [Y/N]"
  echo ">>"

  validResp=0
  while [ $validResp -eq 0 ];
  do
```

```
    read answer
    if [[ "$answer" == "Y" || "$answer" == "y" ]]; then
        echo
        echo "Deleting the existing instance..."
        aws ec2 terminate-instances --instance-ids $existingEc2Instance --region
$region --profile $profile
        #wait for confirmation it was terminated
        aws ec2 wait instance-terminated --instance-ids $existingEc2Instance --region
$region --profile $profile
        validResp="1"
    elif [[ "$answer" == "N" || "$answer" == "n" ]]; then
        echo "Ok, exiting."
        exit 1
    else
        echo "Please reply with Y or N."
    fi
  done

  sleep 10 #give it 10 seconds before trying to delete the SG this instance used.
fi

#check for existing cafeSG security Group
existingMpSg=$(aws ec2 describe-security-groups \
--region $region \
--query "SecurityGroups[?contains(GroupName, 'cafeSG')]" \
--profile $profile | grep GroupId | cut -d "" -f4)

if [[ "$existingMpSg" != "" ]]; then
  echo
  echo "WARNING: Found existing security group with name "$existingMpSg"."
  echo "This script will not succeed if it already exists. "
  echo "Would you like to delete it? [Y/N]"
  echo ">>"

  validResp=0
  while [ $validResp -eq 0 ];
  do
    read answer
    if [[ "$answer" == "Y" || "$answer" == "y" ]]; then
        echo
        echo "Deleting the existing security group..."
        aws ec2 delete-security-group --group-id $existingMpSg --region $region --
profile $profile
        validResp="1"
```

```
    elif [[ "$answer" == "N" || "$answer" == "n" ]]; then
        echo "Ok, exiting."
        exit 1
    else
        echo "Please reply with Y or N."
    fi
  done
  sleep 10 #give it 10 seconds before trying to recreate the SG
fi

# CREATE a security group and capture the name of it
echo
echo "Creating a new security group..."
securityGroup=$(aws ec2 create-security-group --group-name "cafeSG" \
--description "cafeSG" \
--region $region \
--group-name "cafeSG" \
--vpc-id $vpc --profile $profile | grep GroupId | cut -d "" -f4 )
echo "Security Group: "$securityGroup

# Open ports in the security group
echo
echo "Opening port 22 in the new security group"
aws ec2 authorize-security-group-ingress \
--group-id $securityGroup \
--protocol tcp \
--port 22 \
--cidr 0.0.0.0/0 \
--region $region \
--profile $profile

echo "Opening port 80 in the new security group"
aws ec2 authorize-security-group-ingress \
--group-id $securityGroup \
--protocol tcp \
--port 8080 \
--cidr 0.0.0.0/0 \
--region $region \
--profile $profile

echo
echo "Creating an EC2 instance in "$region
instanceDetails=$(aws ec2 run-instances \
--image-id $imageId \
```

```
--count 1 \
--instance-type $instanceType \
--region $region \
--subnet-id $subnetId \
--security-group-ids $securityGroup \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=cafeserver}]'
\
--associate-public-ip-address \
--iam-instance-profile Name=LabInstanceProfile \
--profile $profile \
--user-data file://create-lamp-instance-userdata-v2.txt \
--key-name $key )

#if the create instance command failed, exit this script
if [[ "$?" -ne "0" ]]; then
  exit 1
fi

echo
echo "Instance Details...."
echo $instanceDetails | python -m json.tool

# Extract instanceId
instanceId=$(echo $instanceDetails | python -m json.tool | grep InstanceId | sed -
n 1p | cut -d "" -f4)
echo "instanceId="$instanceId
echo
echo "Waiting for a public IP for the new instance..."
publp=""
while [[ "$publp" == "" ]]; do
  sleep 10;
  publp=$(aws ec2 describe-instances --instance-id $instanceId --region $region --
profile $profile | grep PublicIp | sed -n 1p | cut -d "" -f4)
done

echo
echo "The public IP of your LAMP instance is: "$publp
echo
echo "Download the Key Pair from the Vocareum page."
echo
echo "Then connect using this command (with .pem or .ppk added to the end of
the keypair name):"
echo "ssh -i path-to/"$key" ec2-user@"$publp
echo
```

```
echo "The website should also become available at"
echo "http://"$publp"/cafe/"

echo
DATE=`date '+%Y-%m-%d %H:%M:%S'`
echo
echo "Done running create-instance.sh at "$DATE
```

Mostramos el contenido de los datos de usuario con los que se creó la instancia

```
echo[ec2-user@cli-host starters]$ cat create-lamp-instance-userdata-v2.txt
```

El script de datos de usuario es el siguiente:

```
#!/bin/bash

yum -y update

amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2

yum -y install httpd mariadb-server


systemctl enable httpd

systemctl start httpd


systemctl enable mariadb

systemctl start mariadb


echo '<html><h1>Hello From Your Web Server!</h1></html>' > /var/www/html/index.html

find /var/www -type d -exec chmod 2775 {} \;

find /var/www -type f -exec chmod 0664 {} \;

echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php


usermod -a -G apache ec2-user

chown -R ec2-user:apache /var/www

chmod 2775 /var/www
```

```
#Check /var/log/cloud-init-output.log after this runs to see errors, if any.

#
# Download and unzip the Cafe application files.
#

# Database scripts
wget https://aws-tc-largeobjects.s3.amazonaws.com/CUR-TF-100-RESTRT-1/173-activity-JAWS-troubleshoot-instance/db-v2.tar.gz
tar -zxvf db-v2.tar.gz

# Web application files
wget https://aws-tc-largeobjects.s3.amazonaws.com/CUR-TF-100-RESTRT-1/173-activity-JAWS-troubleshoot-instance/cafe-v2.tar.gz
tar -zxvf cafe-v2.tar.gz -C /var/www/html/

#
# Run the scripts to set the database root password, and create and populate the application database.
# Check the following logs to make sure there are no errors:
#
#     /db/set-root-password.log
#     /db/create-db.log
#
cd db
./set-root-password.sh
./create-db.sh
hostnamectl set-hostname web-server
```

Básicamente en los datos de usuario se instala y activa el servidor Apache, se instala PHP y MariaDB, se descargan los archivos de la página web y se ejecutan los scripts correspondientes para aprovisionar la base de datos.

Ahora ejecutamos el script para lanzar la instancia

```
[ec2-user@cli-host starters]$ ./create-lamp-instance-v2.sh

Running create-instance.sh on 2026-01-23 02:46:35

Instance Type: t3.small
Profile: default

Looking up account values...

VPC: vpc-08954eba0472e0ab5
Region: us-west-2
VPC: vpc-08954eba0472e0ab5
Subnet Id: subnet-0d26578ac460bd37d
Key: vockey
AMI ID: ami-00d8a0260a086a99e

Creating a new security group...
Security Group: sg-02ca034227e658672

Opening port 22 in the new security group
Opening port 80 in the new security group

Creating an EC2 instance in us-west-2

An error occurred (InvalidAMIID.NotFound) when calling the RunInstances operation: The image id '[ami-00d8a0260a086a99e]' does not exist
```

El script falla.

El mensaje indica que el ID de la AMI no existe.

Resolución de problemas

Visualizamos nuevamente el contenido del script

```
[ec2-user@cli-host starters]$ vim create-lamp-instance-v2-sh
```

El problema es que se ha especificado la región como us-east-1, en vez de utilizar la variable $region donde se guardó la región correcta

```
instanceDetails=$(aws ec2 run-instances \
--image-id $imageId \
--count 1 \
--instance-type $instanceType \
--region us-east-1 \
--subnet-id $subnetId \
--security-group-ids $securityGroup \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=cafeserver}]' \
--associate-public-ip-address \
--iam-instance-profile Name=LabInstanceProfile \
--profile $profile \
--user-data file://create-lamp-instance-userdata-v2.txt \
--key-name $key )
```

Corregimos el detalle

```
instanceDetails=$(aws ec2 run-instances \
--image-id $imageId \
--count 1 \
--instance-type $instanceType \
--region $region \
--subnet-id $subnetId \
--security-group-ids $securityGroup \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=cafeserver}]' \
--associate-public-ip-address \
--iam-instance-profile Name=LabInstanceProfile \
--profile $profile \
--user-data file://create-lamp-instance-userdata-v2.txt \
--key-name $key )
```

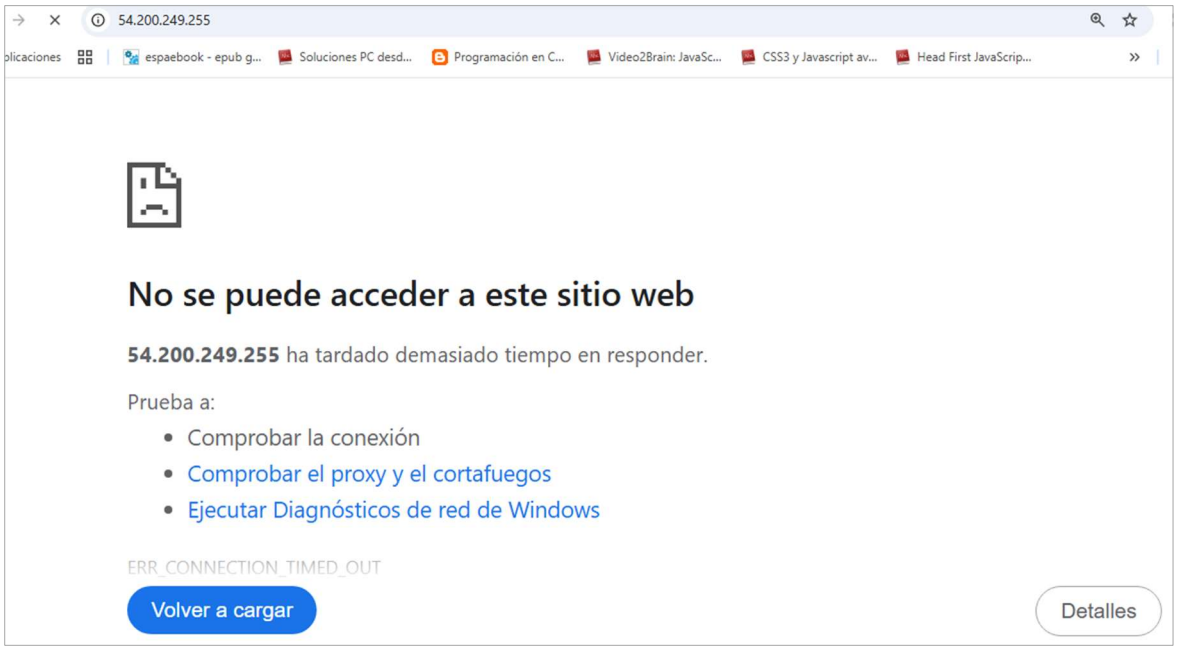
Ejecutamos nuevamente el script

```
[ec2-user@cli-host starters]$ ./create-lamp-instance-v2.sh

Running create-instance.sh on 2026-01-23 03:00:58

Instance Type: t3.small
Profile: default

Looking up account values...

VPC: vpc-08954eba0472e0ab5
Region: us-west-2
VPC: vpc-08954eba0472e0ab5
Subnet Id: subnet-0d26578ac460bd37d
Key: vockey
AMI ID: ami-00d8a0260a086a99e

WARNING: Found existing security group with name sg-02ca034227e658672.
This script will not succeed if it already exists.
Would you like to delete it? [Y/N]
```

```
instanceId=i-0ebc649406104b81b

Waiting for a public IP for the new instance...

The public IP of your LAMP instance is: 54.200.249.255

Download the Key Pair from the Vocareum page.

Then connect using this command (with .pem or .ppk added to the end of the keypair name):
ssh -i path-to/vockey ec2-user@54.200.249.255

The website should also become available at
http://54.200.249.255/cafe/


Done running create-instance.sh at 2026-01-23 03:02:30
```

La instancia ha sido creada.

Ahora accedemos desde el navegador a la dirección IP pública



El sitio web no se muestra

Desde la terminal de la instancia verificamos que el puerto 80 está abierto y que el servidor web está activo

```
[ec2-user@web-server ~]$ sudo netstat -tunlp | grep :80
tcp6       0      0 ::80                    :::*                    LISTEN      2610/httpd
[ec2-user@web-server ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
  Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: active (running) since Fri 2026-01-23 03:03:18 UTC; 6min ago
     Docs: man:httpd.service(8)
 Main PID: 2610 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
   CGroup: /system.slice/httpd.service
           ├─2610 /usr/sbin/httpd -DFOREGROUND
           ├─2617 /usr/sbin/httpd -DFOREGROUND
           ├─2618 /usr/sbin/httpd -DFOREGROUND
           ├─2619 /usr/sbin/httpd -DFOREGROUND
           ├─2620 /usr/sbin/httpd -DFOREGROUND
           └─2621 /usr/sbin/httpd -DFOREGROUND

Jan 23 03:03:18 ip-10-200-0-169.us-west-2.compute.internal systemd[1]: Starting The Apache HTTP Server...
Jan 23 03:03:18 ip-10-200-0-169.us-west-2.compute.internal systemd[1]: Started The Apache HTTP Server.
[ec2-user@web-server ~]$
```

i-0ebc649406104b81b (cafeserver)

PublicIPs: 54.200.249.255 PrivateIPs: 10.200.0.169

Desde la terminal de la instancia CLI Host instalamos *nmap*

```
[ec2-user@cli-host starters]$ sudo yum install -y nmap
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core
Resolving Dependencies
--> Running transaction check
---> Package nmap.x86_64 2:6.40-19.amzn2.0.1 will be installed
--> Processing Dependency: nmap-ncat = 2:6.40-19.amzn2.0.1 for package: 2:nmap-6.40-19.amzn2.0.1.x86_64
--> Running transaction check
---> Package nmap-ncat.x86_64 2:6.40-19.amzn2.0.1 will be installed
--> Finished Dependency Resolution

Dependencies Resolved

================================================================================================
 Package                                Arch                                Version
================================================================================================
Installing:
 nmap                                   x86_64                              2:6.40-19.amzn2.0.1
Installing for dependencies:
```

Nmap (Network Mapper) es una herramienta de código abierto esencial en Linux diseñada para la exploración de redes y la auditoría de seguridad.

Con la opción *-Pn* se hace escaneo de puertos

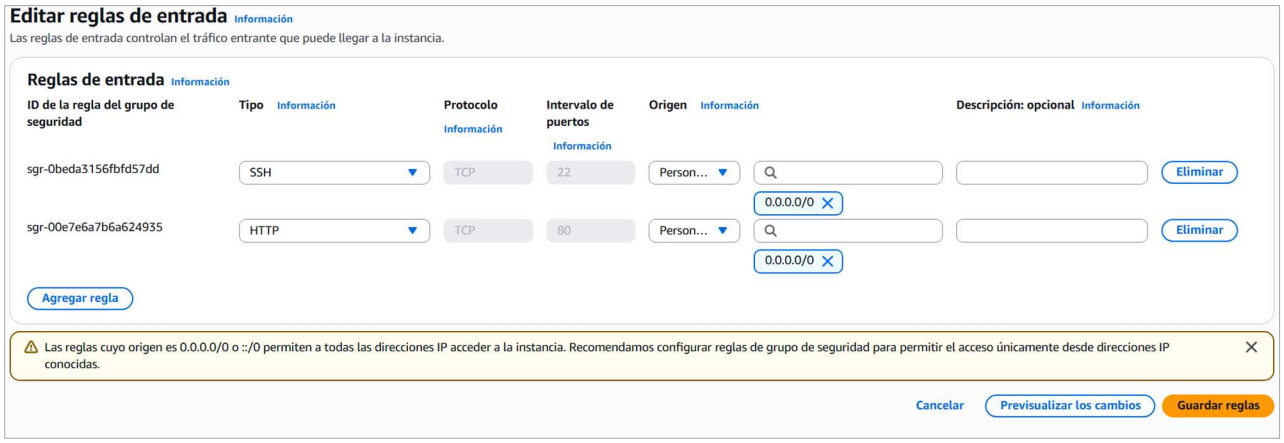
Con el comando *nmap -Pn 54.200.249.255* podemos ver cuales son los puertos accesibles.

Solo el puerto 22 está abierto

```
[ec2-user@cli-host starters]$ nmap -Pn 54.200.249.255

Starting Nmap 6.40 ( http://nmap.org ) at 2026-01-23 03:11 UTC
Nmap scan report for ec2-54-200-249-255.us-west-2.compute.amazonaws.com (54.200.249.255)
Host is up (0.00031s latency).
Not shown: 998 filtered ports
PORT     STATE  SERVICE
22/tcp   open   ssh
8080/tcp closed http-proxy

Nmap done: 1 IP address (1 host up) scanned in 3.93 seconds
```

Con las pruebas podemos concluir que, internamente el servicio de Apache está bien, pero externamente está bloqueado, ya que el puerto 80 ni siquiera aparece como "closed", sino que está filtrado. Esto significa que un firewall está descartando los paquetes antes de que lleguen a Apache.

Para solucionar esto, desde la consola de administración de AWS agregamos una regla de entrada al grupo de seguridad permitiendo el acceso HTTP desde el puerto 80



Ahora la respuesta es exitosa

En las instancias con Linux el servicio *cloud-init* ejecuta los comandos de los datos de usuario.

La función principal del archivo */var/log/cloud-init-output.log* es capturar toda la salida estándar (stdout) y los errores (stderr) que se generan durante la ejecución de los scripts de inicialización cuando una instancia arranca por primera vez.

Ahora vamos a verificar si los datos de usuario al lanzar la instancia se ejecutaron correctamente

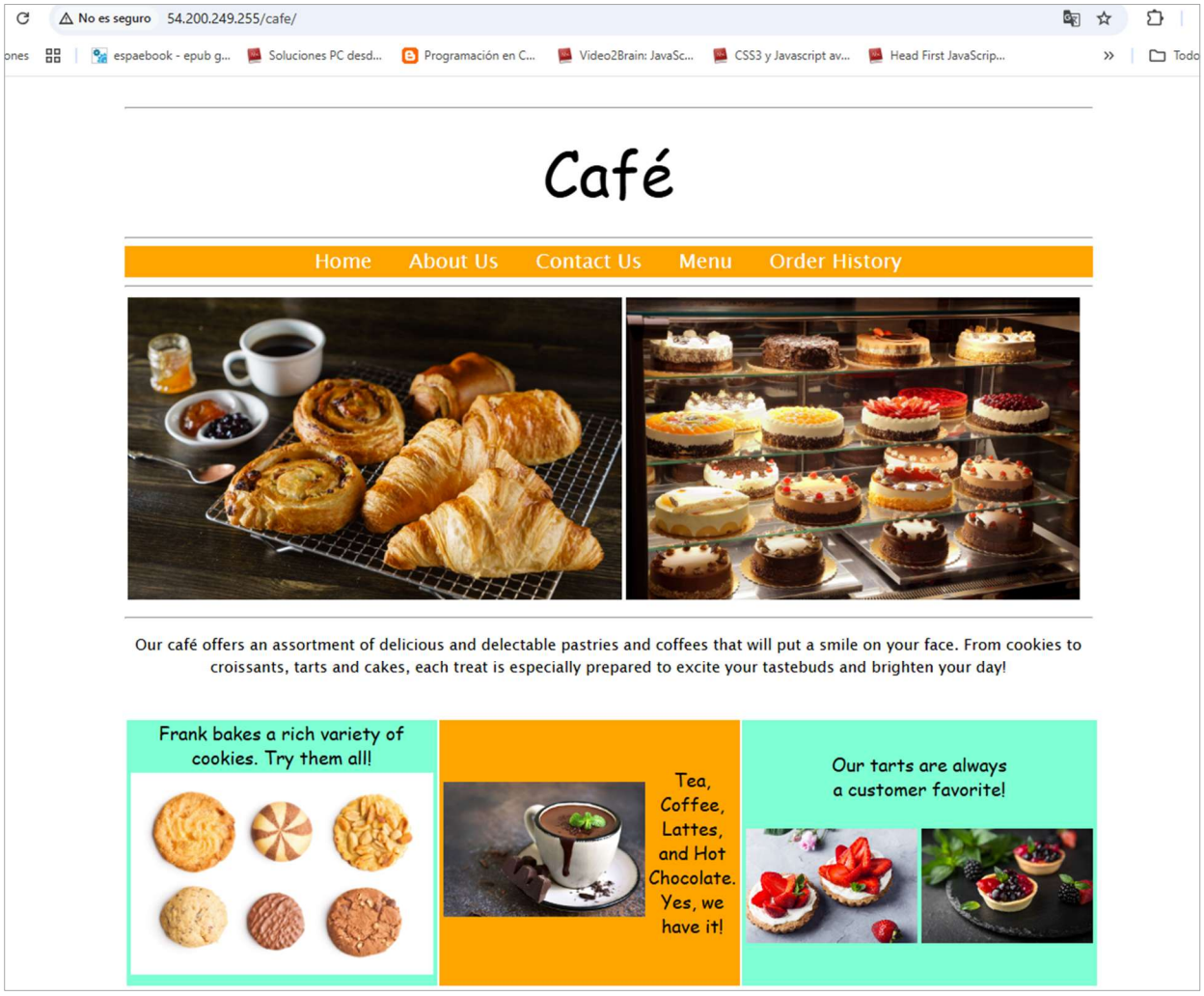
```
[ec2-user@web-server ~]$ sudo tail -f /var/log/cloud-init-output.log
cafe/serverInfo.php

Set Root Password script completed.
Please check the set-root-password.log file to verify successful execution.


Create Database script completed.
Please check the create-db.log file to verify successful execution.

Cloud-init v. 19.3-46.amzn2.0.7 finished at Fri, 23 Jan 2026 03:03:23 +0000. Datasource DataSourceEc2.  Up 58.08 seconds
```

No hay mensajes de error.

Salimos de *tail* con *Ctrl-C*

Verificar la funcionalidad del sitio web

Ahora podemos acceder al sitio web (54.200.249.255/café/) y ordenar postres y bebidas



Café

Home Menu Order History

Order History

Order Number: 2 Date: 2026-01-22 Time: 22:23:35 Total Amount: $6.50

Item	Price	Quantity	Amount
Strawberry Tart	$3.50	1	$3.50
Coffee	$3.00	1	$3.00

Order Number: 1 Date: 2026-01-22 Time: 22:23:12 Total Amount: $8.00

Item	Price	Quantity	Amount
Donut	$1.00	2	$2.00
Hot Chocolate	$3.00	2	$6.00

© 2020, Amazon Web Services, Inc. or its Affiliates. All rights reserved.